

Introduction to React JS

What is React JS?

- ReactJS is a **declarative**, **efficient**, and **flexible** JavaScript library for building reusable UI components.
- It is an **open-source**, **component-based** front end library responsible only for the view layer of the application.

A Brief History

- It was created by Jordan Walke, who was a software engineer at Facebook.
- It was initially developed and maintained by Facebook and was later used in its products like WhatsApp & Instagram.
- Facebook developed ReactJS in 2011 in its newsfeed section, but it was released to the public in the month of May 2013.

Multi Page Application (MPA)

- Traditional way of building websites.
- Each page is a separate HTML file.
- When you click a link → browser requests a new page from the server → entire page reloads.
- **Examples:**
 - E-commerce sites like Amazon
 - Blogs, News websites

Multi Page Application (MPA)

Pros:

- Good for SEO (every page has its own URL & metadata)
- Simpler architecture
- Works well for content-heavy sites

Cons:

- Slower navigation (full reload each time)
- More load on server
- Repeated CSS/JS loading

Single Page Application (SPA)

- Only one HTML file (index.html) is loaded.
- Navigation happens without full page reload.
- Content updates dynamically using JavaScript & AJAX / Fetch API.
- Uses client-side routing (React Router, Angular Router, Vue Router).
- **Examples:**
 - Gmail, Facebook, Twitter
 - React / Angular / Vue Apps

Single Page Application (SPA)

Pros:

- Faster navigation (no reloads)
- Smooth user experience (like desktop apps)
- API-driven → easy integration with backend

Cons:

- SEO challenges (content is loaded dynamically)
- Initial load might be heavier
- Needs client-side JS to work

Why React ?

- Improved Performance thanks to Virtual DOM
- Reusable Components
- Uni-directional Data Flow
- Easy learning curve
- Web and Mobile Apps
- Easy debugging tools

Document Object
Model

Components

- A ReactJS application is made up of multiple components.
- Each component responsible for outputting a small, reusable piece of HTML code.
- These components can be nested with other components to allow complex applications to be built of simple building blocks.

Components (Cont..)

- For example, we take a form that consists of many elements like input fields, labels, or buttons.
- We can write each element of the form as React components, and then we combine it into a higher-level component, i.e., the form component itself.
- The form components would specify the structure of the form along with elements inside of it.

DOM

- DOM stands for Document Object Model.
- It is the structural representation of all nodes in HTML document.
- DOM represents the UI of the application.
- DOM manipulation is required to dynamically change the content of web page.
- The DOM allows scripts (like JavaScript) to manipulate the content, structure, and style of a webpage. For instance, you can dynamically add, remove, or modify elements using JavaScript.

Problem with Direct DOM Manipulation

- Manipulating the DOM directly can be slow and inefficient, especially when dealing with complex or frequently changing UIs.
- The reason is that the DOM is a large, complex data structure, and any changes to it can trigger expensive operations like **reflows** (recalculating the layout of the page) and **repaints** (updating the visuals).
- For example, if you update a large list of elements or change the structure of the page frequently, the browser may struggle to keep up, leading to a sluggish user experience.

Virtual DOM

- **Virtual DOM (VDOM)** is a lightweight JavaScript object representation of the real DOM.
- Instead of touching the real DOM on every update, React:
 - Keeps a copy of the UI in memory (Virtual DOM).
 - Updates the Virtual DOM when state or props change.
 - Compares the new Virtual DOM to the previous one (using diffing).
 - Applies only the necessary changes to the real DOM.
- Think of it as a “blueprint” or “draft copy” of the UI that React works with first, before touching the expensive real DOM.

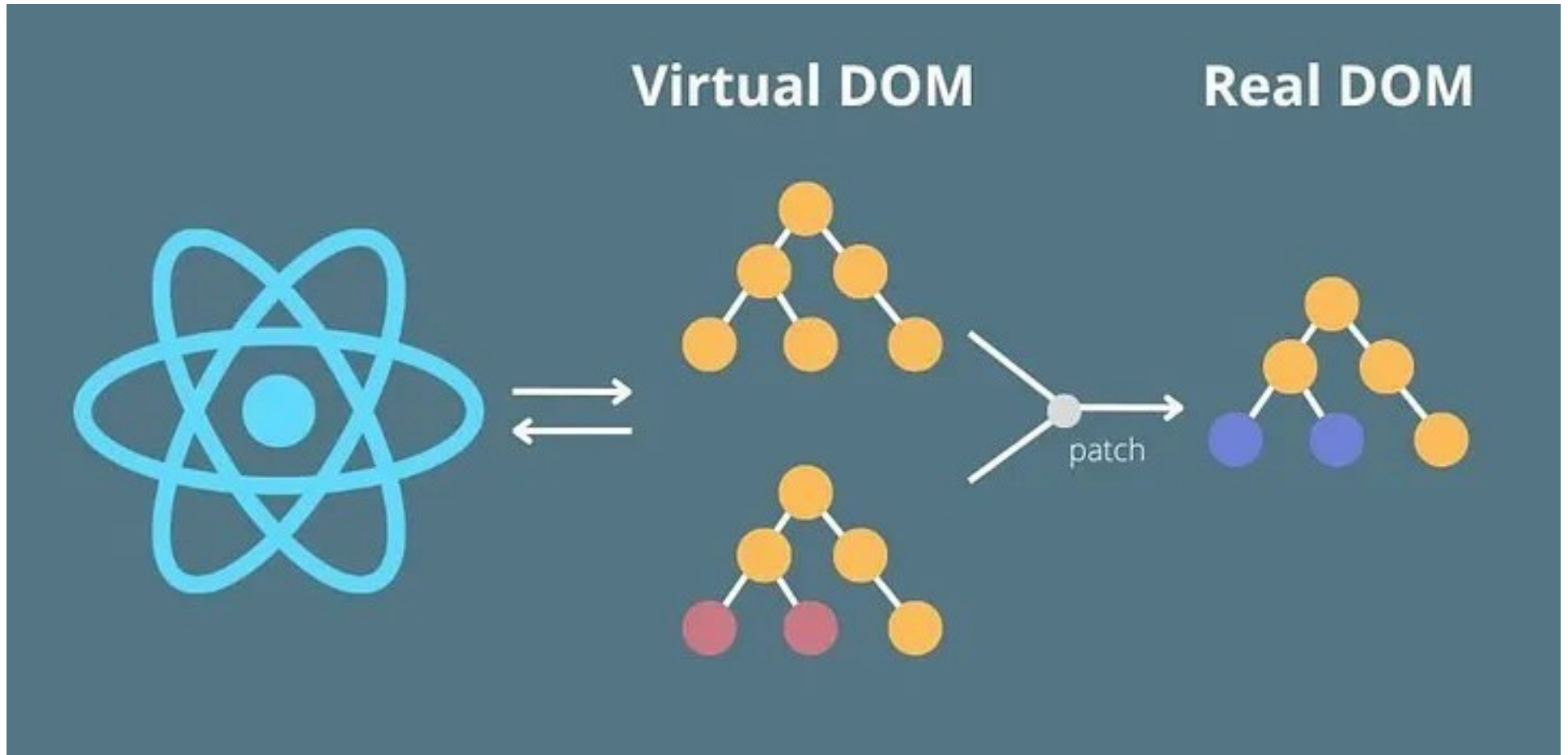
Diffing Algorithm

- React uses a diffing algorithm to efficiently compare the old Virtual DOM with the new one.
- It identifies the differences (or "diffs") between these two representations.
- Once the differences are identified, React creates a minimal set of changes that need to be applied to the real DOM to make it match the new Virtual DOM.

Updating the Real DOM

- Finally, React applies the calculated diffing result to the real DOM, updating only the specific elements that need to change.
- This process is highly optimized and ensures that only necessary changes are made to the actual DOM, minimizing performance bottlenecks.
- This process is also known as "**reconciliation**."

Working of Virtual DOM



Source: <https://javascript.plainenglish.io/react-virtual-dom-the-key-to-efficient-and-optimized-web-development-538f6c502389>

Performance Benefits

- By minimizing the direct manipulation of the real DOM and optimizing updates, React significantly improves the performance of web applications.
- Even though the Virtual DOM introduces some overhead due to its own calculations, the overall gain in performance is substantial, especially in applications with frequent updates.
- React improves perceived performance by batching updates and applying changes efficiently using reconciliation

Modern React Ecosystem

- **Build Tool:**
 - **Vite** (fast dev server & bundler, replaces Create React App)
- **Routing:**
 - **React Router 7** (navigation between pages in SPAs)
- **Styling:**
 - Plain CSS or frameworks like **Tailwind CSS** for rapid UI design
- **State Management:**
 - React Hooks (useState, useReducer, useContext)
 - Advanced: Redux Toolkit or TanStack Query (for larger apps)
- **Package Manager:**
 - npm or yarn (comes with Node.js)

Create and Run ReactJS App

- `npm create vite@latest my-app`

```
◆ Select a framework:  
  ○ Vanilla  
  ○ Vue  
  ● React  
  ○ Preact  
  ○ Lit  
  ○ Svelte  
  ○ Solid  
  ○ Qwik  
  ○ Angular  
  ○ Marko  
  ○ Others
```

```
◆ Select a variant:  
  ○ TypeScript  
  ○ TypeScript + SWC  
  ● JavaScript  
  ○ JavaScript + SWC  
  ○ React Router v7 ↗  
  ○ TanStack Router ↗  
  ○ RedwoodSDK ↗  
  ○ RSC ↗
```

- `cd my-app`
- `npm install`
- `npm run dev`